

AlertaSegura Perú

Guía de instalación y ejecución paso a paso · Explicación de la seguridad implementada

Proyecto: Backend Spring Boot (Java 21+) + Frontend React | **Base de datos:** MySQL | **IDE:** IntelliJ IDEA

- Parte 1 — Requisitos previos
- Parte 2 — Configurar la base de datos MySQL
- Parte 3 — Abrir y correr el backend en IntelliJ IDEA
- Parte 4 — Correr el frontend
- Parte 5 — Probar el flujo completo
- Parte 6 — Ciberseguridad implementada, explicada
- Parte 7 — Solución de problemas comunes

Parte 1 — Requisitos previos

Software que necesitas instalado

Herramienta	Versión mínima	Para qué
Java (JDK)	21	Compilar y correr el backend
Maven	3.8+	Gestionar dependencias del backend (viene integrado en IntelliJ)
MySQL	8.0+	Base de datos del proyecto
Node.js	18+	Correr el frontend (incluye npm)
IntelliJ IDEA	Community o Ultimate	Editar y correr el backend

Verifica antes de empezar: abre una terminal y corre `java -version`, `mysql --version` y `node -version`. Si alguno falla, instálalo antes de continuar.

Parte 2 — Configurar la base de datos MySQL

Paso 2.1 — Crear la base de datos con el schema del proyecto

El proyecto trae un archivo `alertasegura_db.sql` en la raíz con toda la estructura de tablas ya lista. Desde la terminal, en la carpeta del proyecto:

```
mysql -u root -p < alertasegura_db.sql
```

Te pedirá tu contraseña de MySQL. Esto crea la base de datos y todas las tablas necesarias (usuarios, reportes, categorías) automáticamente.

Paso 2.2 — Verificar que se creó correctamente

```
mysql -u root -p -e "USE alerta_segura; SHOW TABLES;"
```

Deberías ver las tablas del proyecto listadas (usuarios, reportes, categorías, y las relacionadas).

Parte 3 — Abrir y correr el backend en IntelliJ IDEA

Paso 3.1 — Abrir el proyecto

1. Abre IntelliJ IDEA
2. `File → Open`
3. Selecciona la carpeta raíz del proyecto (la que contiene `pom.xml` directamente, NO la carpeta `frontend/`)
4. IntelliJ detecta que es un proyecto Maven y pregunta si confías en él — acepta ("Trust Project")

Paso 3.2 — Esperar la indexación

Abajo a la derecha verás una barra de progreso descargando las dependencias (Spring Boot, Spring Security, JWT, Bucket4j, MySQL driver, Lombok, etc.). La primera vez puede tardar varios minutos.

Paso 3.3 — Habilitar Lombok (si IntelliJ lo pide)

El proyecto usa Lombok (genera getters/setters automáticamente). Si IntelliJ muestra un aviso sobre el plugin de Lombok, acéptalo. Si no lo tienes: `File → Settings → Plugins` → busca "Lombok" → instalar → reiniciar IntelliJ.

También activa el "Annotation Processing": `File → Settings → Build, Execution, Deployment → Compiler → Annotation Processors` → marca "Enable annotation processing".

Paso 3.4 — Crear tu archivo de configuración local

El proyecto trae `application-example.properties` como plantilla. Cópialo y renómbralo a `application-dev.properties` , en la misma carpeta (`src/main/resources/`):

```
PORT=8080
DATABASE_URL=jdbc:mysql://localhost:3306/alertasegura
DB_USERNAME=root
DB_PASSWORD=tu_password_real_de_mysql

JWT_SECRET=genera_uno_con_el_comando_de_abajo
JWT_EXPIRATION=86400000
```

Genera un `JWT_SECRET` seguro con este comando en terminal:

```
openssl rand -hex 32
```

Copia el resultado y pégalo como valor de `JWT_SECRET` .

Importante: `application-dev.properties` nunca se sube a Git — ya está en el `.gitignore` del proyecto porque contiene tu contraseña real de MySQL y tu clave secreta.

Paso 3.5 — Configurar el perfil "dev" en IntelliJ

1. Busca la clase con `@SpringBootApplication` (la clase principal, ej. `AlertaseguraApplication.java`)
2. Click derecho → `Run 'AlertaseguraApplication'` (esto crea una configuración de ejecución)
3. Luego ve a `Run → Edit Configurations`
4. En "Active profiles" escribe: `dev`
5. Guarda

Paso 3.6 — Ejecutar

Click en el botón ▶ verde (Run), o `Shift+F10` . En la consola de abajo deberías ver el log de arranque de Spring Boot terminando con algo como:

```
Tomcat started on port 8080
Started AlertaseguraApplication in X.XXX seconds
```

Verificación: abre en el navegador `http://localhost:8080/api/categorias` — deberías ver una respuesta JSON (aunque sea una lista vacía `[]` , significa que el backend está vivo y conectado a la base de datos).

También puedes explorar toda la API con Swagger, ya configurado en el proyecto:

```
http://localhost:8080/swagger-ui/index.html
```

Parte 4 — Correr el frontend

Paso 4.1 — Instalar dependencias

En una terminal nueva (el backend debe seguir corriendo en la otra):

```
cd frontend
npm install
```

Paso 4.2 — Ejecutar

```
npm run dev
```

Verás algo como:

```
→ Local: http://localhost:5173/
```

Abre esa URL en tu navegador.

Nota: el frontend no requiere ninguna configuración adicional para desarrollo local — ya está apuntando a `http://localhost:8080`, que es donde corre el backend por defecto.

Parte 5 — Probar el flujo completo

Flujo de prueba recomendado

1. Abre `http://localhost:5173/register` y crea una cuenta
2. Automáticamente inicia sesión y te lleva al mapa (/)
3. Ve a "Reportar" y crea un reporte de prueba (necesita categoría, distrito, descripción y ubicación en el mapa)
4. Vuelve al mapa — deberías ver tu reporte marcado y en la lista, con estado "PENDIENTE"
5. Para probar la moderación necesitas un usuario con rol MODERADOR — esto se asigna directamente en la base de datos (ver siguiente nota) o mediante el flujo que el equipo haya definido para ascender usuarios

Convertir un usuario en moderador (para probar /moderar)

```
mysql -u root -p -e "USE alertasegura; UPDATE usuarios SET rol='MODERADOR' WHERE email='tu@email.com';"
```

Cierra sesión y vuelve a entrar para que el nuevo rol se refleje en el token JWT.

Parte 6 — Ciberseguridad implementada, explicada

Esta sección explica, con el código real del proyecto, cada capa de seguridad — para que puedan defenderla en una exposición o entrevista con confianza.

6.1 — Autenticación con JWT

Cuando un usuario hace login correctamente, el backend genera un token firmado (JWT) que el frontend guarda y envía en cada petición protegida, en el header `Authorization: Bearer <token>`. El backend verifica la firma del token en cada petición — si alguien lo modifica, la verificación falla y se rechaza con 401.

La clave para firmar (`JWT_SECRET`) vive solo en tu `application-dev.properties` local, nunca en el código ni en Git.

6.2 — Contraseñas hasheadas con BCrypt

Ninguna contraseña se guarda en texto plano. `SecurityConfig.java` define un bean `BCryptPasswordEncoder` que se usa tanto al registrar (para generar el hash) como al hacer login (para comparar sin revertir el hash). BCrypt es lento a propósito — eso lo hace resistente a ataques de fuerza bruta.

6.3 — Política de contraseñas

El registro exige contraseñas de al menos 8 caracteres, con al menos una mayúscula y un número (`RegisterRequestDTO.java` , validado con Bean Validation antes de que la petición llegue a la lógica de negocio):

```
@Size(min = 8, message = "La contraseña debe tener al menos 8 caracteres")
@Pattern(regexp = "^(?=.*[A-Z])(?=.*[0-9]).+$",
        message = "La contraseña debe contener al menos una mayúscula y un número")
```

6.4 — Autorización por rol con @PreAuthorize

No basta con estar autenticado — hay acciones exclusivas de moderadores. El proyecto usa `@PreAuthorize("hasRole('MODERADOR')")` directamente sobre los métodos del controlador que lo requieren:

Endpoint	Restricción
<code>PUT /api/reportes/{id}/estado</code>	Solo MODERADOR
<code>DELETE /api/reportes/{id}</code>	Solo MODERADOR

Si un CIUDADANO intenta llamar a estos endpoints, Spring Security responde 403 (Forbidden) antes de que la petición llegue a la lógica de negocio.

6.5 — Rate limiting contra fuerza bruta y spam (Bucket4j)

`RateLimitFilter.java` limita cuántas peticiones puede hacer una misma IP a los endpoints más sensibles:

Endpoint	Límite	Previene
<code>POST /api/auth/login</code>	5 intentos por minuto	Fuerza bruta de contraseñas
<code>POST /api/reportes</code>	10 por minuto	Spam de reportes falsos

Cuando se supera el límite, el backend responde con código HTTP 429 (Too Many Requests) y un mensaje JSON, sin llegar a procesar la petición.

6.6 — CORS restringido

Solo el frontend, corriendo en `localhost:5173` (o `5174`), puede hacer peticiones al backend desde un navegador. Cualquier otro origen es bloqueado por el navegador antes de que la petición llegue al servidor — configurado explícitamente en `SecurityConfig.java` , no dejado por defecto.

6.7 — Sesiones sin estado (Stateless)

El backend no guarda sesiones en el servidor (`SessionCreationPolicy.STATELESS`). Cada petición se autentica de forma independiente usando el JWT — esto también es por lo que CSRF está desactivado: la protección CSRF tradicional es para sesiones con cookies, y este backend no las usa para autenticación.

6.8 — Protección contra inyección SQL

El backend usa JPA/Hibernate para todo el acceso a datos — las consultas se generan de forma parametrizada automáticamente, nunca concatenando texto del usuario directamente en SQL. Esto cierra la puerta a la inyección SQL clásica sin que el equipo tenga que escribir código defensivo adicional.

6.9 — Endpoints públicos, decididos explícitamente

Por defecto, todo requiere autenticación. Solo se abren explícitamente los que tienen que ser públicos:

- `/api/auth/**` (login, registro — obviamente no pueden requerir estar ya logueado)
- `GET /api/categorias` (cualquiera puede ver las categorías disponibles)
- `GET /api/reportes/**` (el mapa es público, no hace falta cuenta para ver reportes)
- Swagger UI (documentación de la API)

Parte 7 — Solución de problemas comunes

Problema	Causa probable	Solución
El backend no arranca: error de conexión a MySQL	<code>application-dev.properties</code> mal configurado, o MySQL no está corriendo	Verifica que MySQL esté activo y que la contraseña en <code>DB_PASSWORD</code> sea correcta
Error "Schema validation failed"	No se corrió <code>alertasegura_db.sql</code> , o el schema quedó desactualizado	Vuelve a correr <code>mysql -u root -p < alertasegura_db.sql</code>
El frontend no puede conectar con el backend (error de red)	El backend no está corriendo, o está en un puerto distinto a 8080	Confirma que el backend responde en <code>localhost:8080/api/categorias</code>
Error de CORS en la consola del navegador	El frontend está corriendo en un puerto distinto a 5173/5174	Si Vite eligió otro puerto (por estar ocupado el 5173), agrega ese puerto a la lista de <code>allowedOrigins</code> en <code>SecurityConfig.java</code>
Login/registro responde 429 sin haber hecho muchos intentos	El rate limiter cuenta por IP — recargas seguidas durante pruebas pueden activarlo	Espera 1 minuto, el contador se reinicia (ventana fija de Bucket4j)
Lombok: "cannot find symbol" en getters/setters	El plugin de Lombok no está habilitado o el annotation processing está apagado	Ver Paso 3.3 de esta guía

Guía generada para el equipo del proyecto AlertaSegura Perú. Si algo en esta guía no coincide exactamente con el comportamiento real del proyecto tras cambios futuros del equipo, prioricen siempre lo que el código actual haga — esta guía documenta el estado del proyecto al momento de su generación.